

DESIGN AND DEVELOP A REACT JS BASED WEB APPLICATION FOR TICKET BOOKING OF AN INTERNAL DEPARTMENT EVENT

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

K.Gowtham Sai (VTU25540)

10211CS224

Full Stack Application Development

Slot : E1



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "Design and develop a React JS based web application for Ticket Booking of an Internal Department Event" by K.Gowtham Sai (VTU25540) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr.HEMAMALINI.S

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Co-ordinator

Dr.SUMITHRA.M

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

K.Gowtham Sai

Date:06/5/2026

ABSTRACT

The proposed system is a React JS-based web application designed to simplify and modernize ticket booking for internal department events. Traditional manual registration methods often cause delays, errors, and poor data management, so this application provides a streamlined digital solution with a user-friendly interface. It enables students and staff to view upcoming events, access detailed information, check seat availability, and book tickets online with real-time updates and confirmation. Built using React JS, the application ensures fast performance, dynamic content rendering, and reusable components for an enhanced user experience, while optional backend integration with technologies like Node.js and MongoDB supports secure data storage and management. Overall, the system improves efficiency, reduces manual effort, and offers a reliable, transparent, and accessible platform for both event organizers and participants.

Keywords: React JS, Ticket Booking System, Web Application, Event Management, User Authentication, Real-Time Updates, Online Registration, MongoDB, Node.js, User Interface, Data Management

LIST OF FIGURES

3.1	Execution	10
3.2	Structure	12
4.1	Architecture Diagram	13
4.2	DataFlow	14
5.1	Login Page	19
5.2	Events Page	19
5.3	Booking Page	20
5.4	Bookings	20

LIST OF TABLES

7.1	Mapping to Complex Engineering Problems	23
7.2	Mapping to Complex Engineering Activities	24
7.3	SDG Mapping	24

LIST OF ACRONYMS AND ABBREVIATIONS

UI	User Interface
UX	User Experience
API	Application Programming Interface
JS	JavaScript
SPA	Single Page Application
CRUD	Create, Read, Update, Delete
JWT	JSON Web Token
MVC	Model View Controller
DOM	Document Object Model
SQL	Structured Query Language
NoSQL	Non-Relational Database
URL	Uniform Resource Locator

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	2
1.4 Framework	3
2 SURVEY OF SIMILAR APPLICATION IN MARKET	4
3 FRAMEWORK DESCRIPTION	5
3.1 Frontend Implementation	5
3.2 Backend Implementation	7
3.3 Database Implementation	9

4	METHODOLOGIES	13
4.1	Project Architecture	13
4.2	DataFlow Diagram	14
4.3	Module 1:User Management	14
4.4	Module 2:Event Management	15
4.5	Module 3:Booking Management	15
5	RESULTS AND DISCUSSIONS	18
6	CONCLUSION AND FUTURE ENHANCEMENTS	21
6.1	Conclusion	21
6.2	Future Enhancements	22
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	23
7.1	Mapping of the Project to Complex Engineering Prob- lem (CEP)	23
7.2	Mapping of the Project to Complex Engineering Ac- tivities (CEA)	24
7.3	SDG Mapping (CEP Section)	24
	References	24

Chapter 1

INTRODUCTION

1.1 Introduction

In today's digital era, web applications play a crucial role in automating and simplifying organizational processes, including event management. Traditional methods of handling internal department event registrations, such as manual entries or spreadsheets, often result in inefficiencies, errors, and poor data management. To overcome these limitations, the proposed system is a React JS-based web application designed to provide an efficient and user-friendly ticket booking platform. It enables users to view event details, check ticket availability, and book tickets online in a secure and convenient manner. By utilizing React JS, the application ensures a responsive and interactive interface, enhancing the overall user experience. Additionally, integration with backend technologies allows secure storage and management of user and booking data. This system reduces manual workload, im-

proves accuracy and transparency, and offers a reliable digital solution for managing internal department events effectively.

1.2 Aim of the Project

The aim of this project is to design and develop a React JS-based web application for efficient ticket booking of internal department events, providing a user-friendly platform that enables users to view event details, check availability, and book tickets online while ensuring secure data management, real-time updates, and improved accuracy, transparency, and overall event management efficiency.

1.3 Scope of the Project

The scope of this project includes the design and development of a React JS-based web application for managing ticket booking of internal department events in a digital and efficient manner. The system covers functionalities such as user registration and login, viewing event details, checking ticket availability, and booking tickets online with real-time updates. It is intended for use within an organization or academic department, making event management more organized and accessible. The application can be further extended by integrating secure payment gateways, notification systems, and admin controls for

event creation and monitoring. Overall, the project focuses on improving efficiency, reducing manual work, and providing a scalable platform that can be enhanced with additional features in the future.

1.4 Framework

The proposed system is developed using the React JS framework, which is a popular JavaScript library for building fast and interactive user interfaces. React follows a component-based architecture, allowing the application to be divided into reusable and manageable components such as login, event listing, and booking modules. It uses a virtual DOM to improve performance by updating only the required parts of the web page instead of reloading the entire page. The framework supports the development of Single Page Applications (SPA), ensuring smooth navigation and a better user experience. Additionally, React can be integrated with backend technologies like Node.js and databases such as MongoDB to handle data storage, authentication, and booking management efficiently. Overall, the use of React JS enhances scalability, performance, and maintainability of the ticket booking application.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

BookMyShow: Popular platform for booking movie and event tickets Provides real-time seat availability and booking confirmation Supports online payments and digital tickets.

Eventbrite: Used for creating, managing, and booking events Offers event discovery and registration features Provides analytics tools for event organizers.

Ticketmaster: Global ticket booking service for concerts and sports events Includes seat selection and secure payment options Offers mobile ticketing and notifications.

Paytm Insider: Allows booking for events, shows, and experiences Provides offers, discounts, and cashback options Easy integration with digital payments.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

Operating System: Windows / Linux / macOS

Environment: Visual Studio Code (VS Code) or any code editor

Frontend: React JS (Node.js and npm required)

Backend: Node.js with Express.js

Database: MongoDB

Software Requirements:

Install Node.js (includes npm), Install React using npx create-react-app, Install required packages (React Router, Axios, etc.), Install MongoDB for database management (if backend is used)

Hardware Requirements: Minimum 4GB RAM Processor: Intel i3 or above Basic storage for project files Browser Support: Google Chrome / Microsoft Edge / Firefox

3.1.2 Structure of the Project

The React JS-based ticket booking application follows a well-organized and modular project structure to ensure scalability and ease of development. The frontend consists of a public folder containing the main HTML file and a src folder that includes all core functionalities, where assets store images and styles, components contain reusable UI elements like navbar and event cards, pages manage different screens such as home, login, events, booking, and confirmation, services handle API communication, and an optional context folder manages global state like authentication, while App.js and index.js act as the main entry and control files. Additionally, an optional backend built with Node.js and Express.js includes models for database schemas, routes for API endpoints, controllers for handling logic, and configuration files for database connectivity, with a central server file acting as the entry point, making the overall system structured, maintainable, and easy to extend.

3.1.3 Execution Procedure

The execution of the React JS-based ticket booking application begins with setting up the development environment by installing Node.js and required dependencies, followed by creating the React project using appropriate commands and organizing the project structure. After development, the application is run using the npm start command,

which launches it in a web browser for testing and interaction. Users can then access the interface to register or log in, view available events, and book tickets, while the system processes requests and updates data accordingly. If a backend is integrated, the server is started separately to handle API requests and database operations. Finally, the application is tested for functionality, performance, and error handling to ensure smooth operation before deployment.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend environment for the ticket booking application is set up using Node.js and Express.js to handle server-side operations. Initially, Node.js must be installed, which includes npm for managing dependencies. A new backend project is created using npm initialization, and required packages such as Express, Mongoose, CORS, and dotenv are installed. The project is organized with folders for models, routes, controllers, and configuration files. MongoDB is used as the database, which can be set up locally or accessed via cloud services, and connected to the application using Mongoose. Environment variables are configured in a .env file to securely store sensitive information like database URLs and port numbers. The server is then created

using Express and started with a command like `node server.js` or `node-mon` for automatic restarts during development. This setup provides a stable and efficient backend environment for handling API requests, user authentication, and ticket booking data management.

3.2.2 Structure of the Project

The backend of the ticket booking application is organized in a structured and modular way using Node.js and Express.js to ensure scalability and maintainability. The project contains a main root folder that includes essential directories such as models, routes, controllers, and configuration files. The models folder defines the database schemas for entities like users, events, and bookings, which are used to interact with the MongoDB database. The routes folder handles API endpoints for different functionalities such as user authentication, event management, and ticket booking. The controllers folder contains the business logic that processes requests and sends appropriate responses. A config folder is used to manage database connection settings and environment configurations. Additionally, a main server file (`server.js`) acts as the entry point of the application, where the Express server is initialized and middleware is configured. The project also includes a `package.json` file for managing dependencies and scripts. This structured approach helps in organizing code efficiently, making it easier to develop, test, and extend the backend application.

3.2.3 Execution Procedure

The execution of the backend application begins with setting up the Node.js environment and installing all required dependencies using npm. After organizing the project structure, the MongoDB database connection is configured using Mongoose and environment variables are defined in a .env file. The Express server is then initialized in the main server file, where middleware and API routes are configured. Once the setup is complete, the backend server is started using commands such as `node server.js` or `nodemon server.js` for automatic restarts during development. The server begins listening on a specified port and handles incoming API requests from the frontend, performing operations like user authentication, event management, and ticket booking. Finally, the application is tested using tools like browsers or API testing tools to ensure proper functionality and smooth communication between frontend and backend systems.

3.3 Database Implementation

3.3.1 Database Configuration The database configuration for the ticket booking application is carried out using MongoDB with the help of Mongoose, which provides an easy way to interact with the database in a Node.js environment. Initially, it installed locally or ac-



Figure 3.1: Execution

cessed through a cloud service, and a new database is created to store application data such as users, events, and bookings. The connection to the database is established by creating a configuration file (commonly named `db.js`) where Mongoose is imported and used to connect to the MongoDB URI stored in environment variables. The connection string is securely managed using a `.env` file to protect sensitive information. Once connected, schemas and models are defined to structure the data, and these models are used throughout the application to perform operations like creating, updating, and deleting records. Proper error handling and connection status messages are included to ensure smooth

functioning of database within the backend system [Figure3.1].

3.1.2 Schema Structure

The schema structure of the ticket booking application is designed using MongoDB with Mongoose to define and organize the data in a structured manner. The system mainly consists of three core schemas: User, Event, and Booking. The User schema stores details such as user ID, name, email, password, and role for authentication and authorization purposes. The Event schema contains information related to events, including event ID, title, description, date, time, venue, total seats, and available seats. The Booking schema manages ticket booking details such as booking ID, user ID (reference), event ID (reference), number of tickets booked, booking date, and status. Relationships between these schemas are maintained using references, where the Booking schema links users and events to ensure data consistency. This structured schema design enables efficient data storage, retrieval, and management, supporting all core functionalities of the ticket booking system.

3.1.3 Tables created

The ticket booking application uses a database structure that consists of three main tables (or collections in MongoDB) to manage all the required data efficiently. The User table is created to store user-related information such as user ID, name, email, password, and role,

which is used for authentication and access control. The Event table stores details of all events, including event ID, event name, description, date, time, venue, total seats, and available seats, allowing users to view and select events. The Booking table is used to record ticket booking information, which includes booking ID, user ID (as a reference), event ID (as a reference), number of tickets booked, booking date, and booking status. These tables are interconnected, where the Booking table links both User and Event tables to maintain relationships and ensure data consistency. This structure helps in efficient data management, retrieval, and smooth functioning of the application.

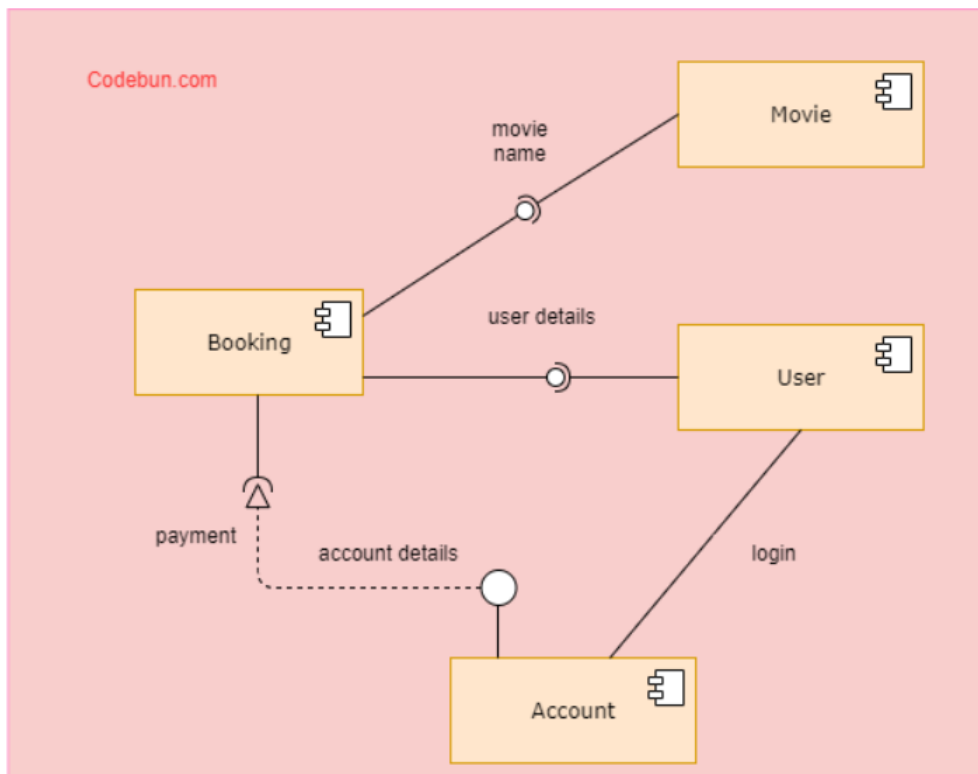


Figure 3.2: Structure

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The overall system architecture of the ticket booking application is shown. It illustrates the interaction between frontend, backend, and database components [Figure 4.1].

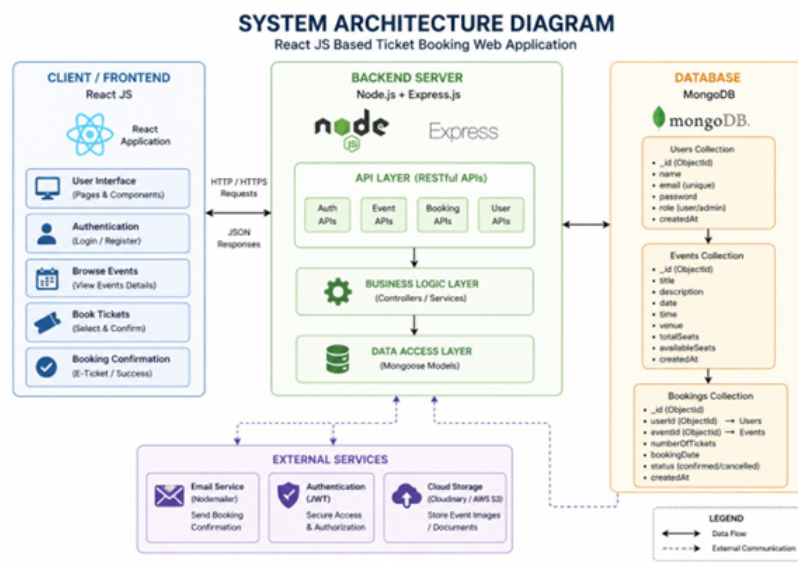


Figure 4.1: Architecture Diagram

4.2 DataFlow Diagram

The data flow of the system is depicted and showing how data moves across different modules [Figure 4.2].

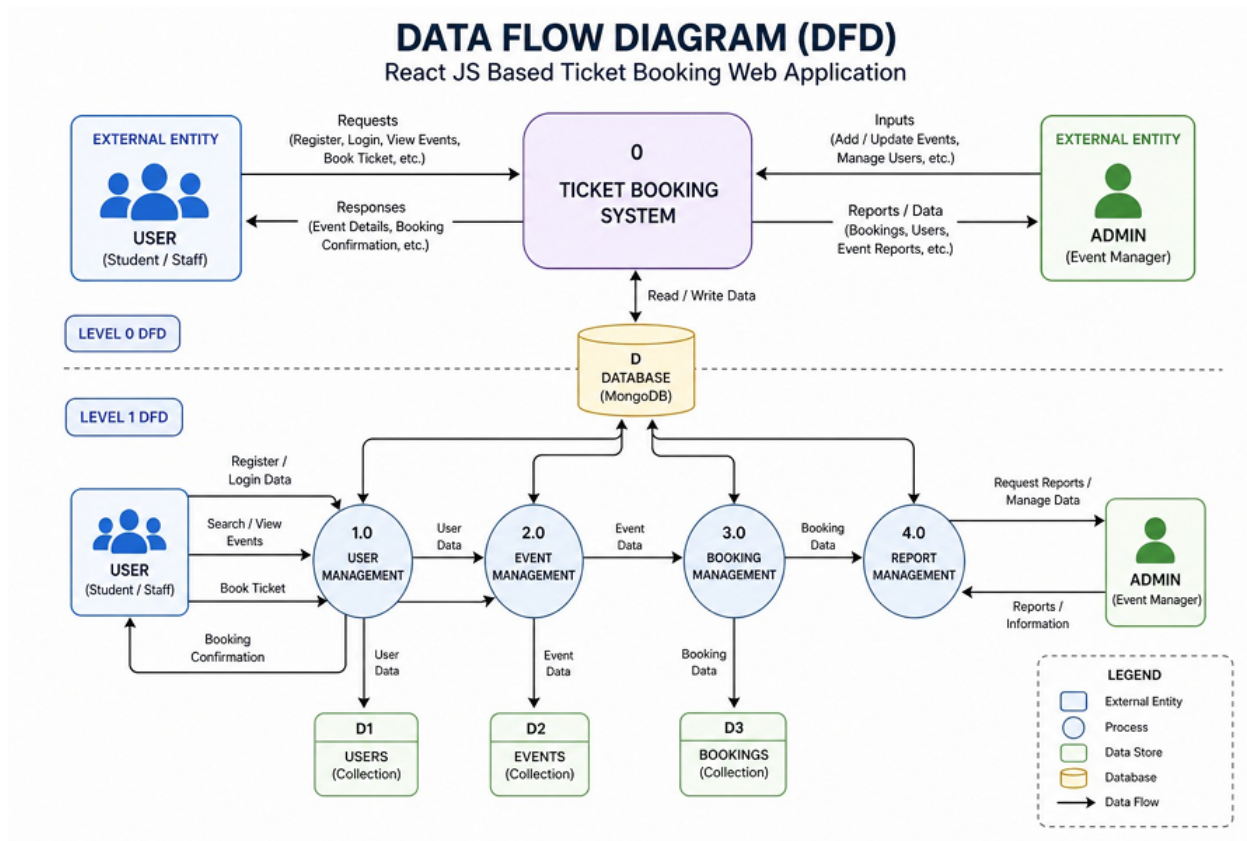


Figure 4.2: DataFlow

4.3 Module 1:User Management

This module handles all user-related functionalities in the ticket booking system. It allows users such as students and staff to register and create an account by providing necessary details like name, email, and password. Once registered, users can log in securely using their credentials, and authentication mechanisms ensure that only au-

thorized users can access the system. The module also manages user sessions and roles, such as admin and normal user, to provide appropriate access permissions. Additionally, users can update their profile details and view their booking history. This module plays a crucial role in maintaining security and personalized access within the application.

4.4 Module 2:Event Management

The event management module is responsible for handling all event-related operations in the system. It allows administrators to create, update, and delete event details such as event name, description, date, time, venue, and number of available seats. Users can view a list of available events along with detailed information and select events based on their preferences. The module ensures real-time updates of event data, especially seat availability, to prevent overbooking. It also helps in organizing events efficiently and provides a structured way to manage all event information within the application.

4.5 Module 3:Booking Management

The booking management module manages the complete ticket booking process. It allows users to select an event, choose the number of tickets, and confirm their booking. Once a booking is made, the sys-

tem updates the available seats automatically and generates a booking record with details such as booking ID, user information, event details, and booking status. The module also provides booking confirmation to users and allows them to view their booking history. In case of cancellation (if implemented), the system updates the seat availability accordingly. This module ensures smooth and accurate handling of all booking transactions in the application.

Advantages of this Project:

User-Friendly Interface: Provides a simple and interactive interface for easy navigation and booking.

Time Saving: Reduces manual effort and allows quick ticket booking.

Real-Time Updates: Shows live event details and seat availability.

Improved Accuracy: Minimizes human errors in booking and data handling.

Secure Data Management: Ensures safe storage of user and booking information.

Accessibility: Can be accessed anytime and from anywhere via web browsers.

Scalability: Can be extended with additional features like payments and notifications.

Reduced Paperwork: Eliminates the need for physical records and forms.

Efficient Event Management: Helps organizers manage events effectively.

Cost Effective: Reduces operational and administrative costs.

Disadvantages:

Internet Dependency: Requires a stable internet connection to access and use the application.

Initial Development Cost: Development and deployment may require time and resources.

Security Risks: Vulnerable to cyber threats if proper security measures are not implemented.

Technical Issues: System bugs or server downtime may affect performance.

Learning Curve: Some users may find it difficult to use initially if they are not familiar with web applications.

Maintenance Required: Requires regular updates and maintenance for smooth functioning.

Data Privacy Concerns: User data must be handled carefully to avoid misuse.

Chapter 5

RESULTS AND DISCUSSIONS

The developed React JS-based ticket booking application was successfully implemented and tested across all functionalities, providing a user-friendly interface for users to register, log in, view events, and book tickets efficiently. All modules, including user management, event management, and booking management, function correctly and deliver accurate results. The system was tested under various scenarios, where event details were displayed properly, and the booking process updated seat availability in real time, preventing overbooking. User authentication ensured secure access, and booking confirmations were generated successfully after each transaction. The integration between the frontend and backend operated smoothly, enabling seamless data flow and storage. Overall, the system demonstrated improved efficiency, reduced manual effort, minimized errors, and provided better data management, confirming that it meets the project objectives effectively.

The login interface of the application is shown in Fig. 5.1, which allows users to securely access the system.

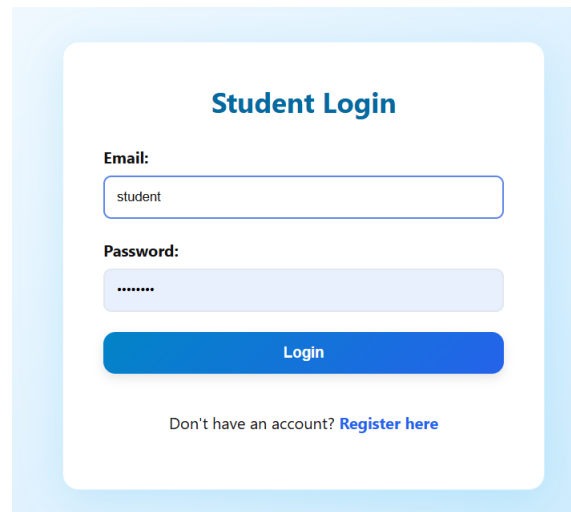
The image shows a 'Student Login' form. It has a title 'Student Login' in blue. Below it, there are two input fields: 'Email:' with the text 'student' and 'Password:' with masked characters '*****'. A blue 'Login' button is positioned below the password field. At the bottom, there is a link that says 'Don't have an account? Register here'.

Figure 5.1: Login Page

The events page is displayed in Fig. 5.2, where users can view available events along with their details.

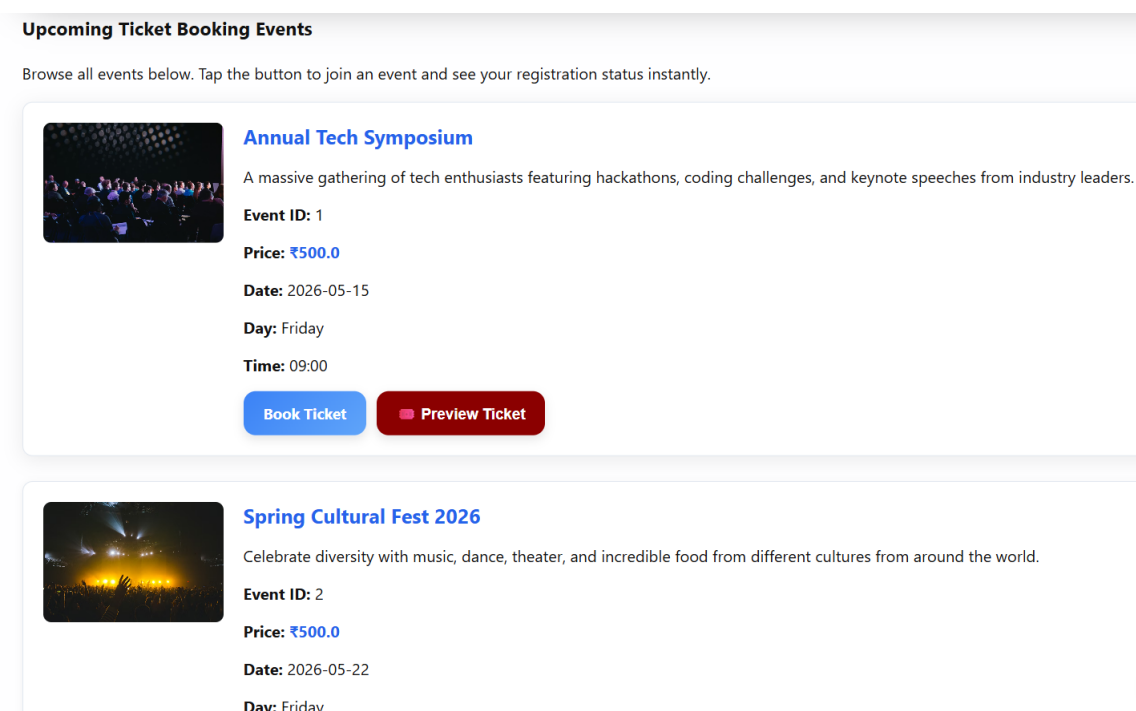
The image displays a section titled 'Upcoming Ticket Booking Events'. Below the title is a subtitle: 'Browse all events below. Tap the button to join an event and see your registration status instantly.' There are two event cards. The first card is for the 'Annual Tech Symposium', featuring a photo of a group of people. It includes details: 'Event ID: 1', 'Price: ₹500.0', 'Date: 2026-05-15', 'Day: Friday', and 'Time: 09:00'. At the bottom of the card are two buttons: 'Book Ticket' (blue) and 'Preview Ticket' (red). The second card is for the 'Spring Cultural Fest 2026', featuring a photo of a night festival. It includes details: 'Event ID: 2', 'Price: ₹500.0', 'Date: 2026-05-22', and 'Day: Friday'.

Figure 5.2: Events Page

The booking interface is shown in Fig. 5.3, enabling users to select tickets and confirm their booking.

Ticket Checkout

Cancel

Event Details



Title

Global Career Fair

Date & Day

2026-06-15 (Monday)

Time

11:00

Location

TBA

Student Information

Name

gowtham

Email

vtu25538@veltech.edu.in

Department

ece

Phone

N/A

Total Amount to Pay

Free

Secure payment processed internally

Confirm & Book Ticket

Figure 5.3: Booking Page

The booking records are presented in Fig. 5.4, where users can view their confirmed bookings.



Robotics Showcase

Witness autonomous drones and combat rovers built by the engineering department in full action in the central courtyard.

Event ID: 9

Price: Free

Date: 2026-07-20

Day: Monday

Time: 09:30

Joined

Preview Ticket



DIGITAL TICKET PREVIEW

Includes: PDF Ticket, Gmail Confirmation, & Entry QR Code.

Book now to receive your official ticket via email.

Figure 5.4: Bookings

20

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The React JS-based ticket booking application for internal department events has been successfully designed and developed, providing an efficient and user-friendly solution for managing event registrations and ticket bookings. The system overcomes the limitations of traditional manual methods by offering features such as online event viewing, real-time seat availability, and secure booking processes. It ensures accuracy, reduces time consumption, and improves overall data management through seamless integration between frontend and backend technologies. The modular structure of the application makes it scalable and easy to maintain, enhancements and feature additions. Overall, the project achieves its objective of delivering a reliable, transparent, and accessible platform for both users and event organizers.

6.2 Future Enhancements

The proposed ticket booking application can be further improved by incorporating advanced features to enhance functionality and user experience. Future enhancements may include the integration of secure online payment gateways to enable seamless transactions, implementation of notification systems such as email or SMS alerts for booking confirmations and event updates, and development of a dedicated admin dashboard for better event management and analytics. Additional features like QR code-based digital tickets, seat selection visualization, and cancellation/refund options can improve usability. The system can also be extended to support mobile applications, multi-language interfaces, and cloud deployment for better scalability and accessibility. These enhancements will make the application more robust, flexible, and suitable for real-world large-scale usage.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

The mapping of the project to complex engineering problems is presented in [Table 7.1], which highlights different attributes and their justification in the system.

Table 7.1: Mapping to Complex Engineering Problems

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	Requires in-depth engineering	WK4, WK5	Knowledge of React JS, hooks, validation, UI design
WP2	Conflicting Requirements	Technical and non-technical constraints	WK4, WK5	Balances usability with booking and validation
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4	Component design, state handling, dynamic updates
WP4	Familiarity	Partially new problems	WK4	Real-time booking and ticket updates
WP5	Applicable Codes	Limited standard solutions	WK6	Custom booking and validation logic
WP6	Stakeholder Needs	Multiple stakeholders	WK5, WK6	Students, faculty, organizers
WP7	Interdependence	System-level integration	WK3, WK5	UI + state + validation integration

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

The alignment of the project with complex engineering activities is shown in [Table 7.2], which describes the use of technologies and system interactions.

Table 7.2: Mapping to Complex Engineering Activities

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of technologies	React JS, HTML, CSS
EA2	Level of Interactions	Complex module interaction	Event display + booking + validation
EA3	Innovation	Modern technologies	Hooks, conditional rendering
EA4	Consequences	Impact on users	Simplifies booking
EA5	Familiarity	Beyond standard	Dynamic real-time system

7.3 SDG Mapping (CEP Section)

The relationship between the project and Sustainable Development Goals (SDGs) is summarized in [Table 7.3], showing how the system contributes to societal and technological development.

Table 7.3: SDG Mapping

SDG No.	SDG Title	Relevance
SDG 4	Quality Education	Supports academic events
SDG 9	Innovation	Promotes React-based systems
SDG 10	Reduced Inequalities	Accessible to all users
SDG 11	Sustainable Communities	Encourages organized events

References

- [1] BookMyShow Official Website. <https://in.bookmyshow.com>
- [2] Paytm Insider Official Website. <https://insider.in>
- [3] Ticketmaster Official Website. <https://www.ticketmaster.com>
- [4] Eventbrite Official Website. <https://www.eventbrite.com>
- [5] StubHub Official Website. <https://www.stubhub.com>